

Smart Recipe Search Engine Architecture

1. Project Overview

Smart Recipe Search Engine is an Information Retrieval based system that allows users to search recipes using ingredients and retrieve the most relevant recipes.

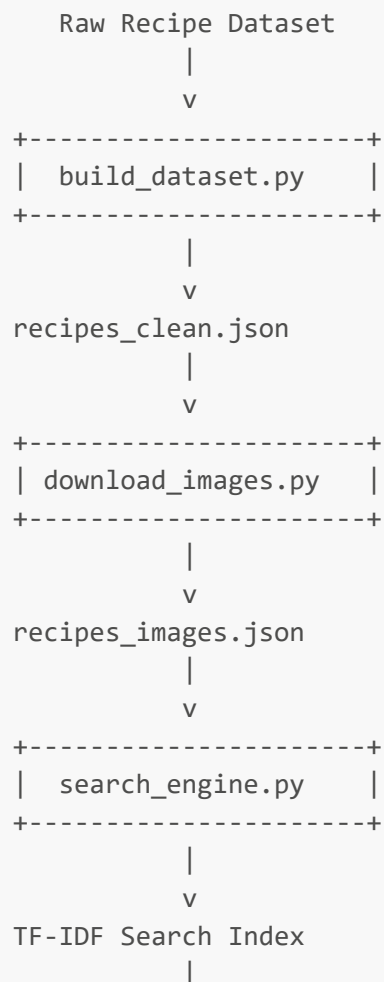
The system uses:

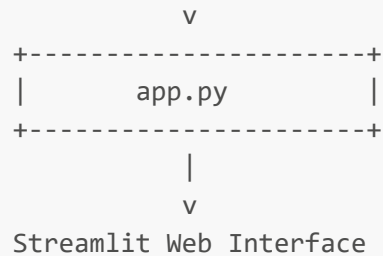
- TF-IDF text representation
- Cosine Similarity
- Popularity-based ranking
- Ingredient matching
- Calorie filtering

The final application is implemented using Streamlit and provides an interactive user interface for recipe discovery.

2. High-Level Architecture

The complete workflow of the system:





3. Project Components

3.1 build_dataset.py

Purpose

Responsible for cleaning and preprocessing the raw recipe dataset.

Responsibilities

- Loading recipe data from Parquet files
- Removing invalid recipes
- Filtering recipes based on:
 - Reviews
 - Ratings
 - Ingredients
 - Instructions
 - Images
- Extracting recipe metadata
- Processing ingredients
- Processing cooking instructions
- Extracting cuisine information
- Processing calorie information

Input

```
data/recipes.parquet
```

Output

```
data/recipes_clean.json
```

3.2 download_images.py

Purpose

Downloads recipe images and connects them with recipe records.

Responsibilities

- Reading image URLs from cleaned recipes
- Downloading images using multiple workers
- Saving images locally
- Creating image filenames
- Removing unused image URL information

Input

```
data/recipes_clean.json
```

Output

```
data/images/  
data/recipes_images.json
```

3.3 search_engine.py

Purpose

The core Information Retrieval engine of the project.

Responsibilities

- Loading processed recipes
- Creating TF-IDF representation
- Building vocabulary
- Calculating document frequency
- Creating recipe vectors
- Calculating similarity scores
- Ranking recipes
- Applying calorie filtering

Input

```
data/recipes_images.json
```

Generated Cache

```
cache/index.pkl  
cache/vocabulary.json
```

4. Information Retrieval Pipeline

4.1 Text Preprocessing

Before searching, text goes through preprocessing:

```
Raw Text  
  |  
  v  
Tokenization  
  |  
  v  
Lowercase Conversion  
  |  
  v  
Stopword Removal  
  |  
  v  
Manual Stemming  
  |  
  v  
Processed Tokens
```

4.2 TF-IDF Model

Each recipe is represented as a numerical vector.

The system uses:

- Recipe title
- Ingredient names
- Instructions
- Keywords

The importance of each word is calculated using TF-IDF.

TF-IDF allows the system to identify important words inside recipes.

4.3 Similarity Calculation

User queries are converted into TF-IDF vectors.

Similarity between query and recipes is calculated using Cosine Similarity.

```
Similarity = Cosine(Query Vector, Recipe Vector)
```

4.4 Ranking System

The final ranking score combines:

```
Final Score =  
0.9 * Similarity Score  
+  
0.1 * Popularity Score
```

Similarity has higher importance because matching user ingredients is the main goal.

5. Data Flow

Dataset Preparation Flow

```
recipes.parquet  
  |  
  v  
Cleaning  
  |  
  v  
Filtering  
  |  
  v  
Feature Extraction  
  |  
  v  
recipes_clean.json
```

Image Processing Flow

```
recipes_clean.json  
  |  
  v  
Extract Image URLs  
  |  
  v  
Download Images  
  |  
  v  
Local Image Storage  
  |
```

```
v
recipes_images.json
```

Search Flow

```
User Query
  |
  v
Query Preprocessing
  |
  v
TF-IDF Vector Creation
  |
  v
Similarity Calculation
  |
  v
Ranking
  |
  v
Top-K Recipes
```

6. Application Layer

app.py

The frontend application is developed using Streamlit.

It provides:

- Ingredient search box
- Number of results selection
- Calorie range filtering
- Recipe cards
- Recipe images
- Ingredient details
- Cooking instructions
- Ranking information

7. Cache System

vocabulary.json

Stores the generated vocabulary from the TF-IDF model.

Purpose:

- Faster processing
- Vocabulary reference

index.pkl

Stores the generated search index.

Contains:

- Processed recipes
- TF-IDF matrix
- IDF values

Purpose:

- Avoid rebuilding the search index every time
- Improve application startup speed

Note:

This file is generated automatically and should not be uploaded to GitHub.

8. Directory Organization

```
Recipe_Search_Engine/  
  
├── app.py  
├── build_dataset.py  
├── download_images.py  
├── search_engine.py  
  
├── data/  
│   ├── recipes_clean.json  
│   └── recipes_images.json  
  
├── cache/  
│   └── vocabulary.json  
  
├── screenshots/  
  
├── docs/  
│   └── architecture.md  
  
├── README.md  
└── requirements.txt
```

9. Execution Workflow

Complete execution:

1. Install dependencies

|
v

2. Run `build_dataset.py`

|
v

3. Run `download_images.py`

|
v

4. Run `app.py` using Streamlit

|
v

5. Search and retrieve recipes

10. Future Improvements

Possible improvements:

- Using advanced embedding models
- Neural search methods
- Better ranking algorithms
- User personalization
- Recipe recommendation system
- Database integration
- Cloud deployment

Author

Sheyda Fathi